# Computer software

## Operating system

Page No.
Date

- Hardware refers to physical device of computer system.

Software refers to set of computer programs, procedures, and associated documents (flowchart, manual etc) describing the program, and how they are to be used.

Sequence of instructions written in a language understood by a computer is called computer program.

A software package is a group of programs that solve a specific problem or perform a specific type of job.

Relation b/w hardware & software

- both hardware & software are nuessary for computer to do useful job. Both are complementary to each other.
- some hardware can be loaded with different software to make a computer perform different types of jobs
- Except for updates, hardware is normally one time expense, software is continuing expense.

Types of software → system software
→ Application software

(1) System software

- set of one or more programs designed to control the operation and extend processing capability of computer system.
- system software makes operation of computer system more effective and efficient.
- It provides support for development and execution of application software (programs).
- The programs included in system software package are called system programs.

Some common types of system software are

(i) Operating system

(ii) Programming Language Translators

(iii) Communication software

(iv) Utility programs : help user to maintain system and in performing task of routine nature.
eg: formatting of hard disk or floppy disk, backup of files etc.

Application software

- Set of one or more programs designed to solve specific problem or task.
- programs included in application software package are called application programs.

ex:

(i) Word-Processing software – MS word

(ii) Spreadsheet software – MS Excel

(iii) Database software – MS Access

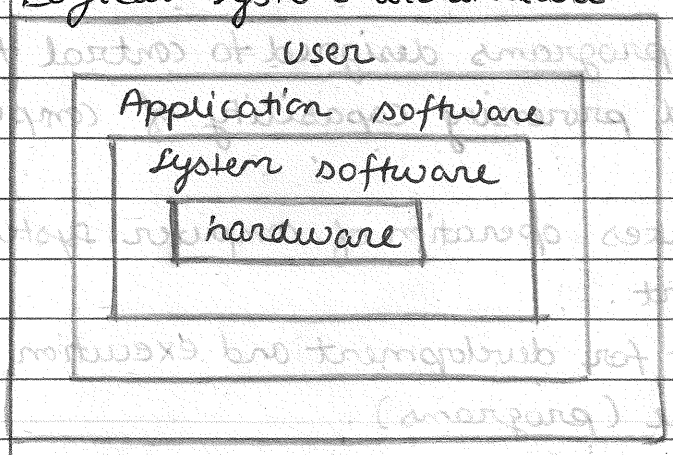
(iv) Graphic software – MS Paint

(v) Personal Assistance software

(vi) Education software

(vii) Entertainment Software

Logical system architecture



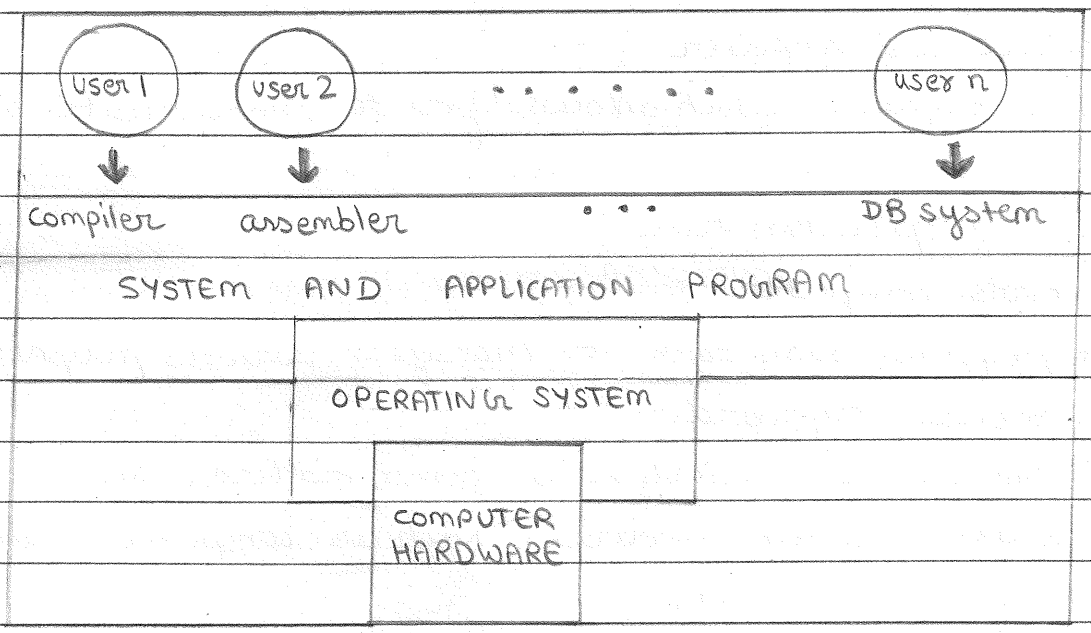

It depicts relationship among hardware, system software, application software and user of computer system.

# Operating System

Dt.:
Pg.:

A computer has many resources (hardware and software) which may be required to complete a task.
The commonly required resources are I/O devices, memory, file storage space, CPU etc. operating system acts as a manager of above resources and allocates them to specific programs and users as necessary for their task.
Operating system is a resource manager.

computer system




Two views of operating system
User view: of computer refers to the interface being used.
Such system are designed for one user to monopolize its resources, to maximize the user is performing the work. In there cases, Operating system is designed mostly for ease of use, with some attention paid to performance, and none paid to resource utilization

Dt.:
Pg.:

System view: operating system can be viewed as a resource allocator. The operating system acts as a manager of resource, decides b/w conflicting requests, control execution etc.

operating system management task.

(i) Processor management: which involves putting tasks into order and pairing them into manageable size before they go to CPU
(ii) memory management: which coordinates data to and from RAM and determines necessity for virtual memory
(iii) Device management: which provides interface b/w connected devices
(iv) Storage management: which directs permanent data storage.
(v) Application: which allows standard communication b/w software and computer.
(vi) User interface: which allows you to communicate with computer

Function of operation task.

(i) It boots computer
(ii) It performs basic task ie. managing various peripheral devices eg. mouse, keyboard
(iii) It provides user interface ie command line, GUI
(iv) It handles system resources such as computer's memory and sharing of CPU time
(v) It provides file management which refers to the way that operating system manipulates, stores, retrieves and save data
(vi) Error handling is done by OS.

BIOS → ROM

Dt.:
Pg.:

# Types of Operating System

(i) simple batch system.

There is no direct interaction b/w user and computer. User has to submit a job to computer operator. The computer operator places a batch of several jobs on an input device. Jobs are batched together by type of languages and requirement. Then a special program, the monitor, manages the execution of each program in batch. The monitor is always in main memory and available for execution.

Disadvantages
- Zero interaction b/w user and computer
- no mechanism to prioritize processes

(ii) multiprogramming batch system.

In this, O.S picks and begins to execute one job from memory. Once this job needs an I/O operation, O.S switch to another job (CPU and OS always busy). Job in memory are always less than no. of jobs on disk (Job pool). If several jobs are ready to run at same time, then system chooses which one to run (CPU scheduling). In non-multiprogrammed system, there are moments when CPU sits idle and does not do any work. In multiprogramming system, CPU will never be idle and keeps on processing.

Time-sharing system are an extension of multiprogramming systems. In time sharing systems the prime focus is on minimizing response time, while in multiprogramming the prime focus is to maximise CPU usage.

→ time is shared among n-users on a CPU.

(iii) multiprocessor systems.

A multiprocesor system consist of several processors (CPU) that share a common physical memory. multiprocessor system provides higher computing speed and power. In multiprocesor system all processors operate under single operating system. multiplicity of processors and how they do act together are transparent to each other.

advantage
- enhanced performance
- execution of several tasks by different processors concurrently increases system's throughput without speeding up execution of single task
- If possible, system divides task into many subtask and then subtask can be executed in parallel in different processors. (Single task execution speedup)

(iv) Distributed operating system

compromises of many computers that are interconnected by communication network. The main benefit of distributed system is its low price/performance ratio.

advantage
- as there are multiple system involved, user at one site can utilize resources of system at other sites for resource-intensive tasks.
- fast processing
- less load on host machine

(v) Real-time operating system

- known to give maximum time for critical operations (like OS calls, interrupt handling) and complete them on time are referred to as Hard-Real time operating system.
- that guarantee a maximum of time ie. critical task

Dt.: Pg.: Delta

will get priority above other tasks, but no assurity of completing it in a defined time are referred to as soft-Real time operating system.

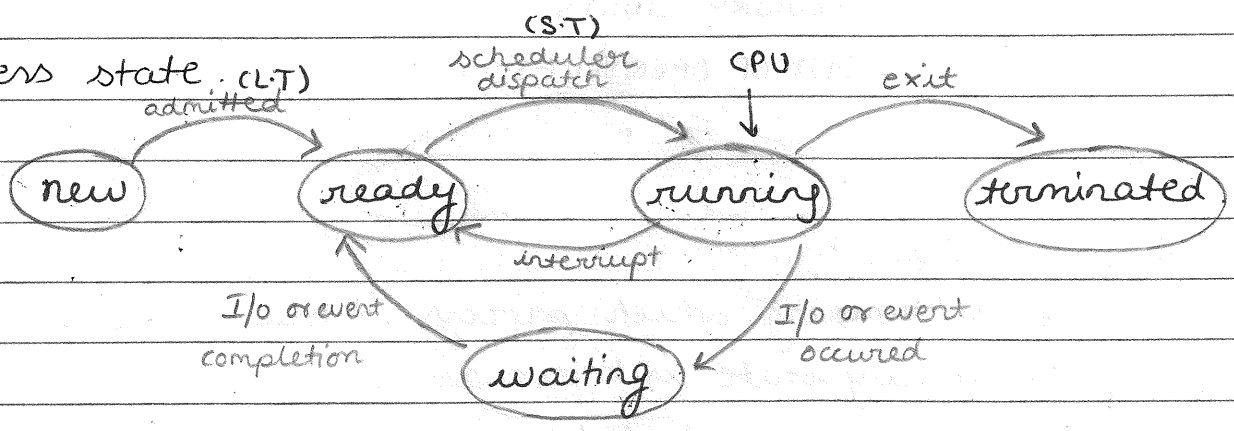
## Process and PCB

Process : a program in execution is called a Process. Program and Process are different. A process is an 'active' entity as opposed to program which is considered to be 'passive' entity. Attributes held by process include hardware state, memory, CPU etc.

Process memory is divided into four section -

- text section is made up of compiled program code, read in from non-volatile storage when program is launched.
- data section is made up of global and static variables, allocated and initialized prior to executing main
- heap is used for dynamic memory allocation, and is managed via calls to new, delete, malloc, free etc.
- The stack is used for local variables. Space on stack is reserved for local variables when they are declared.

Process state




(i) new - process is in stage of being created. (ii) ready - process has all resources that it needs to run but CPU is busy on another instruction (iii) running - CPU is working on this process. (iv) waiting - process is waiting for some resource or event to occur. (v) Terminated - process has completed.

Dt.:
Pg.:

## Process control Block (PCB)

There is a PCB for each process enclosing all information about process. It is a data structure, which contains:

(i) Process state - It can be running, waiting etc.

(ii) Process ID and parent process ID

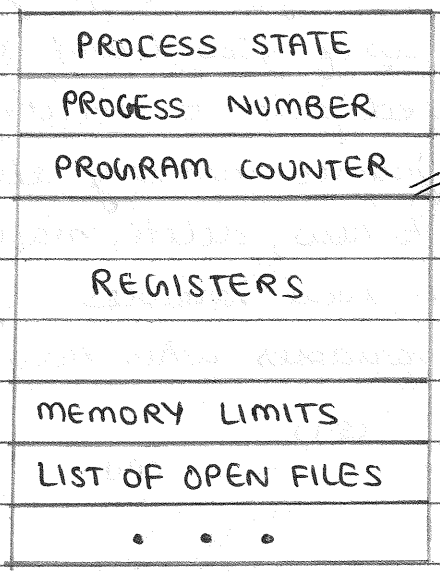
(iii) CPU registers and Program counter - P.C holds address of next instruction to be executed for that process

(iv) CPU scheduling information - such as priority information and pointers to scheduling queues.

(v) Memory management information - eg: page table or segment tables

(vi) Accounting information - user and kernel CPU time consumed, account numbers, limit etc.

(vii) I/o status information - device allocated, open file tables etc.




## PROCESS SCHEDULING

The act of determining which process in ready queue should be moved to running state is known as process scheduling

Prime aim of process scheduling system is to keep CPU busy all time and to deliver minimum response time for all programs.

Dt.: Pg.: Delta

schedulers fell into one of two general categories:

Non pre-emptive scheduling - when currently executing process gives up CPU voluntarily.

Pre-emptive scheduling - when operating system decides to favour another process, pre-empting the currently executing process ie. forced deallocation of CPU from a program that is currently running on CPU.

scheduling queues.

- All processes when enters into system are stored in job queue.
- processes in ready state are placed in ready queue
- processes waiting for a device to become available are placed in device queue. There are unique device queue for each I/O device available.

Types of scheduler

(i) Long term scheduler - runs less frequently.
removes processes from main memory to reduce degree of multiprogramming
- decides which program must get into job queue. - Job scheduler
- puts job from new state to ready state
- aim to maintain good degree of multiprogramming

(ii) Short term scheduler - runs very frequently
- also known as CPU scheduler
- puts job from ready state to running state.
- increases process execution rate and enhance CPU performance.

(iii) Medium term scheduler - during extra load, this scheduler picks out big processes from ready queue for some time, to allow smaller processes to execute, thereby reducing no. of processes in ready queue.

Dt.: Pg.: Delta

## CPU Scheduling

is a process which allows one process to use the CPU while the execution of another process is on hold (in waiting state) due to unavailability of any resource like I/O etc, thereby making full use of CPU.
The aim of CPU scheduling is to make system efficient, fast and fair.

## Scheduling criteria

(i) CPU utilization: To make best use of CPU and not to waste any CPU cycle.

(ii) Throughput: It is total no. of processes completed per unit time. This may range from 10/sec to 1/hr

(iii) Turnaround time: amount of time taken to execute a particular process ie. interval from time of submission (arrival time) to time of completion.

(iv) Waiting time: time spent waiting in ready queue to acquire get control on CPU

(v) Load average: is average number of processes residing in ready queue waiting for their turn to get into the CPU

(vi) Response time: it is the amount of time taken when a request was submitted (arrival time) to the first response it produced.

In general, CPU utilization and throughput are maximized and other factors are reduced for proper optimization

complete time of each process ie end.
turnaround time = completion - arrival time.
waiting time = turnaround - burst time

turnaround time = completion time − arrival time

Dt.: Pg.: Delta

Scheduling Algorithms (mechanism)

(1) First come first serve (FCFS) scheduling

- Jobs are executed on first come, first serve basis
- easy to understand and implement
- poor in performance as average wait time is high.

| Process | AT | Burst time |
|---|---|---|
| $P_1$ | | 21 |
| $P_2$ | | 3 |
| $P_3$ | | 6 |
| $P_4$ | | 2 |

GANTT chart



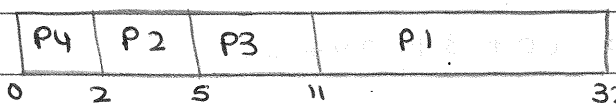

no starvation.

average waiting time = (0 + 21 + 24 + 30)/4

= 18.75 ms

(11) Shortest Job First (SJF) scheduling

- best approach to minimize waiting time
- actual time taken by process is already known to processor
- impossible to implement.




GANTT chart

average waiting time = (0 + 2 + 5 + 11)/4

= 4.5 ms

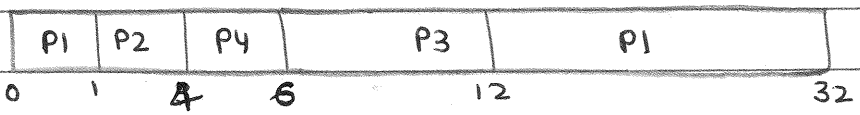
It is optimal. Gives minimum waiting time.

Preemptive shortest Job First

| Process | burst time | arrival time |
|---|---|---|
| P1 | 21 | 0 |
| P2 | 3 | 1 |
| P3 | 6 | 2 |
| P4 | 2 | 3 |

Starvation.

GANTT chart




average waiting time = $(5-3)+(6-2)+(12-1))/4$

$= 4.25$ ms

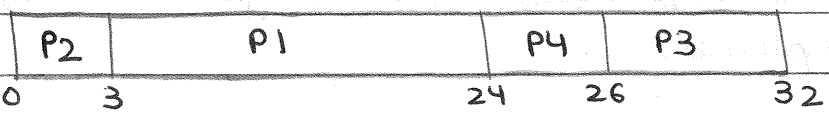
(iii) Priority scheduling

- priority is assigned for each process
- process with highest priority is executed first and soon.
- processes with same priority are executed in FCFS.
- priority can be decided based on memory requirements, time requirements or any other resource requirement.

| Process | burst time | priority |
|---|---|---|
| P1 | 21 | 2 |
| P2 | 3 | 1 |
| P3 | 6 | 4 |
| P4 | 2 | 3 |

starvation may occur. hence aging is used to solve this

GANTT chart




average waiting time = $(0+3+24+26)/4 = 13.25$ ms.

(iv) Round Robin (RR) scheduling.

- A fixed time is allocated to each process, called quantum for execution
- one process is executed for given quantum and then process is preempted and other process executes for another quantum.
- context switching is used to save states of preempted processes

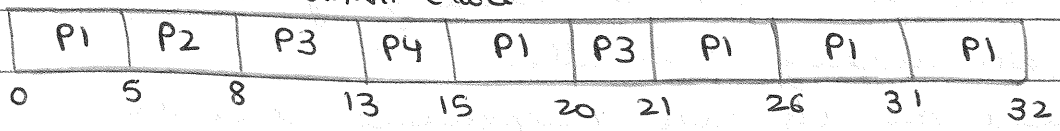
no starvation.

Dt.:
Pg.:

| Process | burst time |
|---|---|
| P1 | 21 |
| P2 | 3 |
| P3 | 6 |
| P4 | 2 |

here quantum, $q = 5$

GRANTT chart




average waiting time = 11 ms.

(v) multilevel queue scheduling

- multiple queues are maintained for processes
- each queue can have its own scheduling algorithms.
- priorities are assigned to each queue.

Foreground process, high priority → RR. $q_1$ burst time <= 2 ms

$q_2$ 2 < burst time ≤ 5 ms

Background process, low priority → FCFS. $q_3$ 5 < burst time ≤ n ms

depending on priority each queue will be completed.

Let $q_1$ has highest priority, then $q_2$ and then $q_3$

So, $q_2$ will executed only if $q_1$ is finished

$q_3$ " " " " $q_1$ and $q_2$ are finished.

(vi) multilevel feedback queue.

- what if jobs are coming again and again and $q_3$ process never gets chance.

$q_1 = 2$ ms

$q_2 = 5$ ms

$q_3 = 10$ ms

- job will always enter queue $q_1$.

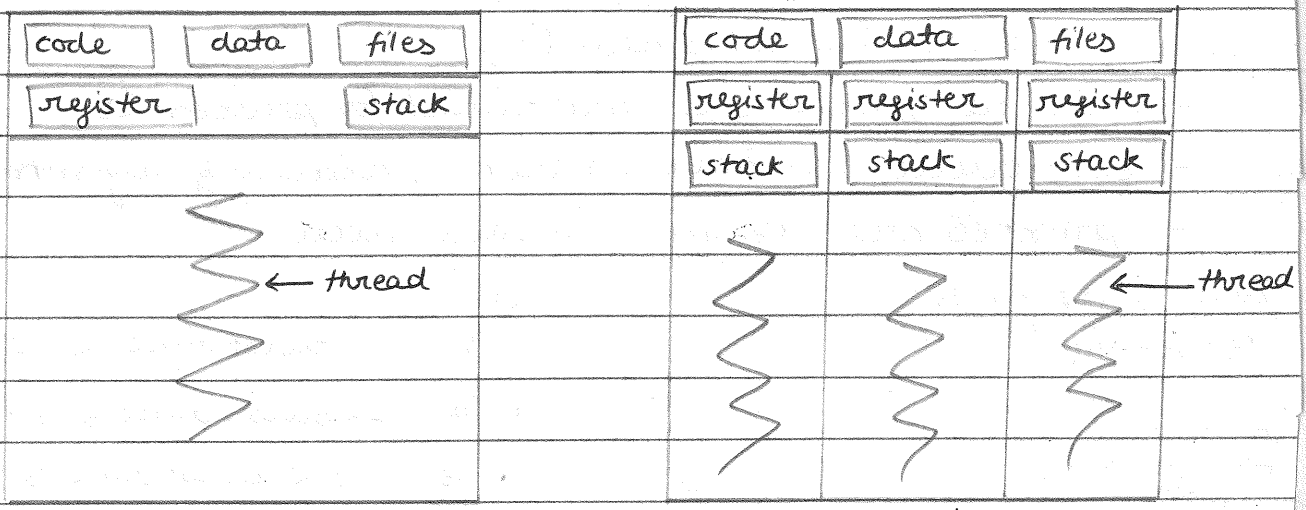
if job $P_1$ enters $q_1$, it will execute 2 ms. If it finishes in 2 ms, it will remain in queue $q_1$ else it will be pushed to queue $q_2$. and same goes on.

Dt.:
Pg.:

## Threads

Thread is an execution unit which consist of its own program counter, a stack and a set of registers. Threads are also known as Lightweight process. Threads are popular way to improve application through parallelism. The CPU switches rapidly back and forth among the threads giving illusion that the threads are running in parallel.

As each thread has its own independent resource for process execution, multiple processes can be executed parallely by increasing no. of threads.




single-threaded process  multithreaded process

## Types of Thread

User threads - PID = 1
are above kernel and without kernel support. These are threads that application programmers use in their programs.

Kernel threads - PID = 0.
are supported within kernel of OS itself. All modern OSs support kernel level threads, allowing kernel to perform multiple simultaneous tasks and/or to service multiple kernel system calls simultaneously.

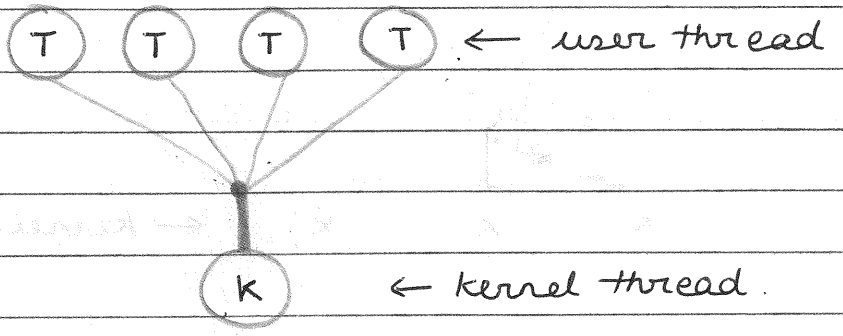
Dt.:
Pg.:

## Multithreading Models

The user threads must be mapped to kernel threads by one of the following strategies -

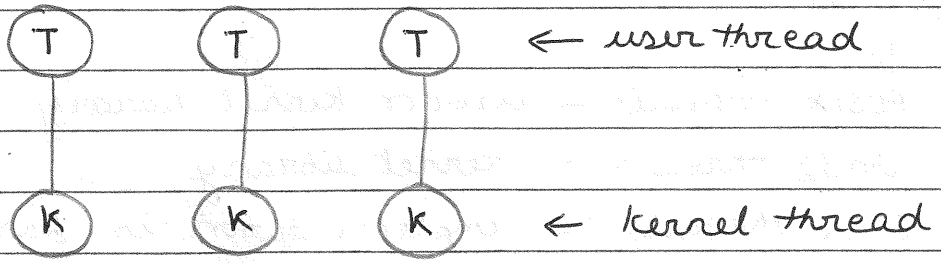
(i) Many - To - One model

- many user level threads are all mapped onto a single level thread
- thread management is handled by thread library in user space, which is efficient in nature.



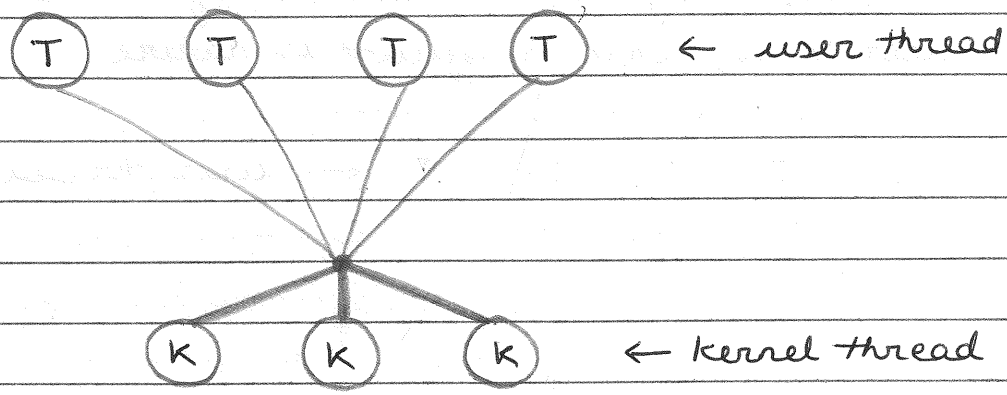

(ii) One - to - One model

- creates a separate kernel thread to handle each and every user thread
- most implementations of this model place a limit on how many threads can be created
- linux and windows from 95 to XP implement one - to - one model for threads.



(iii) Many - To - many Model

- this model multiplexes any no. of user threads onto an equal or smaller no. of kernel threads.
- user can create any no. of threads.
- blocking the kernel system calls doesnot block the entire process
- processes can be split across multiple processors.




Thread Libraries

provides ~~programm~~ programmers with API for creating and managing of threads.

Thread libraries may be implemented either in user space or in kernel space.

The user space involves API functions implemented solely within user space, with no kernel support. The kernel space involves system calls and require a kernel with thread library support.

Three types of thread:

- POSIX Pitheads – user or kernel library
- Win32 threads – kernel library
- Java threads – implementation is based upon OS and hardware JVM is running on.

Dt.:
Pg.:

Benifits of multithreading

(i) Responsiveness

(ii) Resource sharing

(iii) Economy

(iv) scalability : threads can be distributed over a series of processor

(v) Context switching is smooth.

Multithreading Issues.

(i) Thread Cancellation: means terminating a thread before it has finished working. Two approaches for this are

- Asynchronous cancellation - terminates target thread immediately
- Deferred cancellation - allows target thread to periodically check if it should be cancelled

(ii) Signal Handling: multithreaded process recieved a signal, to which thread it must be delivered? It can be delivered to all, or a single thread

(iii) fork() system call : fork() is a system call executed in kernel through which a process creates a copy of itself. Now problem is, if one thread forks, will entire process be copied or not?

(iv) Security Issues: because of extensive sharing of resources b/w multiple threads.

Dt.:
Pg.:

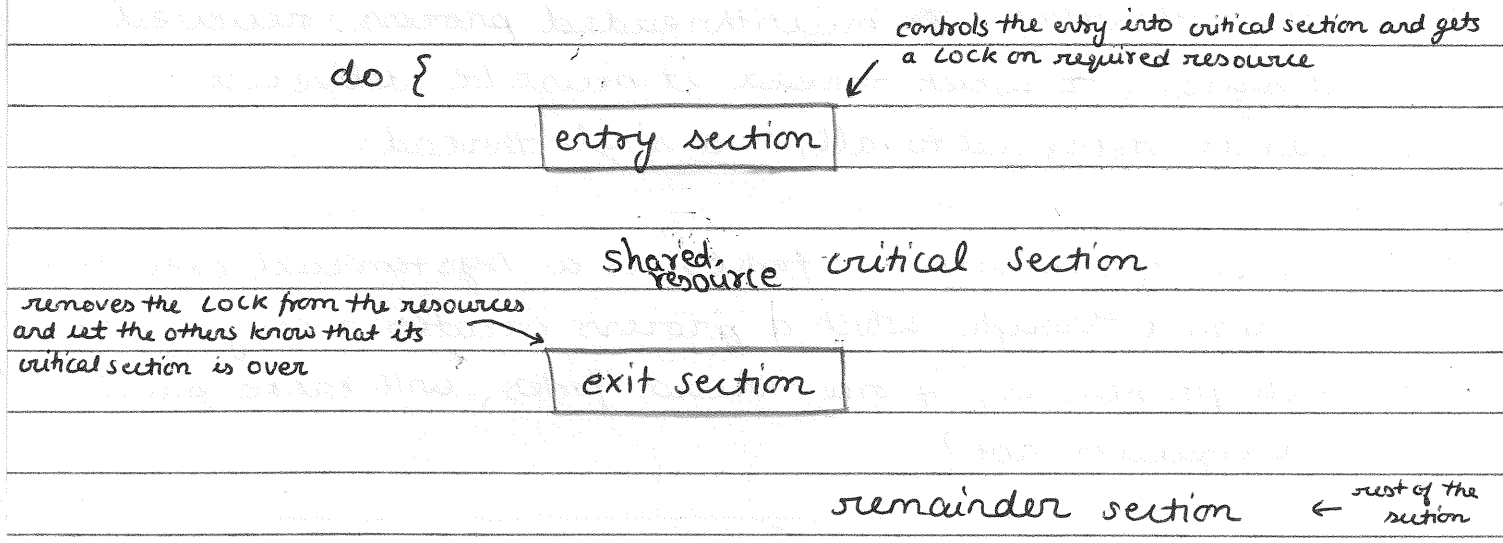
## Process synchronization.

means sharing system resources by processes in such a way that, Concurrent access to shared data is handled thereby minimizing the chance of inconsistent data. Maintaining data inconsistency demands mechanisms to ensure synchronized execution of cooperating processes.

Process Synchronization was introduced to handle problems that arose while multiple process executions.

## critical section Problem (for shared resource)

A critical section is a code segment that accesses shared variables and has to be executed as an atomic action. It means that in a group of cooperating processes, at a given point of time, only one process must be executing its critical section. If any other process also wants to execute its critical sections, it must wait until the first one finishes.




## Solution to critical section problem

A solution to critical section problem must satisfy three cond$^n$:

(1) Mutual Exclusion - out of group of cooperating processes, only one process can be in its critical section at a given point of time. No other process is allowed to enter critical section.

(ii) Progress: If no process is in its critical section, and if one or more threads wants to execute their critical section then any one of the threads must be allowed to get into its critical section. Process who enter C.S must be taken by those who wanted to enter C.S and should be taken in finite time.

(iii) Bounded Waiting: waiting should be finite. When a process has put an request to enter CS, after that no. of processes to enter C.S and came out of ~~request~~ C.S before that request is enter must be finite.

Synchronization Hardware (Hardware approach) Test & set

Many systems provide hardware support for critical section code.

The critical section problem can be easily in a single-processor environment if we could ~~allow~~ disallow interrupts to occur while a shared variable is being modified. In this manner, we can say that current sequence of instructions would be allowed to execute in order without preemption.

Unfortunately, this solution is not feasible in a multiprocessor environment. Disabling interrupts on a multiprocessor environment can be time consuming as the message is passed to all the processor.

This message transmission lag, delays entry of threads into critical section and system efficiency decreases.

Mutex Locks (software approach)

As synchronization hardware is not easy to implement, a strict software approach called Mutex Lock was introduced.

In this section of code, a Lock is aquired over the critical resources modified and used inside critical section and in exit section Lock is released. As the resource is locked while a process executes in critical section hence no other process can aces it.

Dt.: Pg.: Delta

## Semaphores (software solution)

In 1965, Dijkstra introduced (proposed) a new and very significant technique for managing concurrent processes by using a value of simple integer variable to synchronize the process of interacting processes. This integer variable is called semaphore. So it is basically a synchronizing tool and is accessed only through two low standard atomic operation, wait ie P() and ~~Signal~~ Signal ie. V() respectively.

The classical definition of wait and signal are:

Wait: decrement the value of its argument S as soon as it would become non-negative

Signal: increment value of its argument, S as an individual operation

→ shared variable is S

```
P(S)                          V(S)
{ while S≤0 doskip;           {
    S = S-1;                      S = S+1;
}                             }
```

```
Pi = P(S)
     critical section
     V(S)
     remainder section.
```

Race condition: When order of execution can change result.

Properties of Semaphores

(i) Simple

(ii) Works with many processes

(iii) Can have many different critical sections with different semaphores

(iv) Each critical section has unique access semaphores

(v) Can permit multiple processes into critical section at once, if desirable.

Dt.: Pg.: Delta

Types of semaphores.

(i) Binary semaphore : It is a special form of semaphore used for implementing mutual exclusion, hence it is often called Mutex. A binary semaphore is intialized to 1 and only takes the value 0 and 1 during execution of program.

(ii) Counting semaphore : These are used to implement bounded concurrency.

Limitations of Semaphore

(i) Priority inversion is a big limitation of semaphores.

(ii) Their use is not enforced, but is by conversion only

(iii) with improper use, a process may block indefinitely. Such a process is called deadlock.

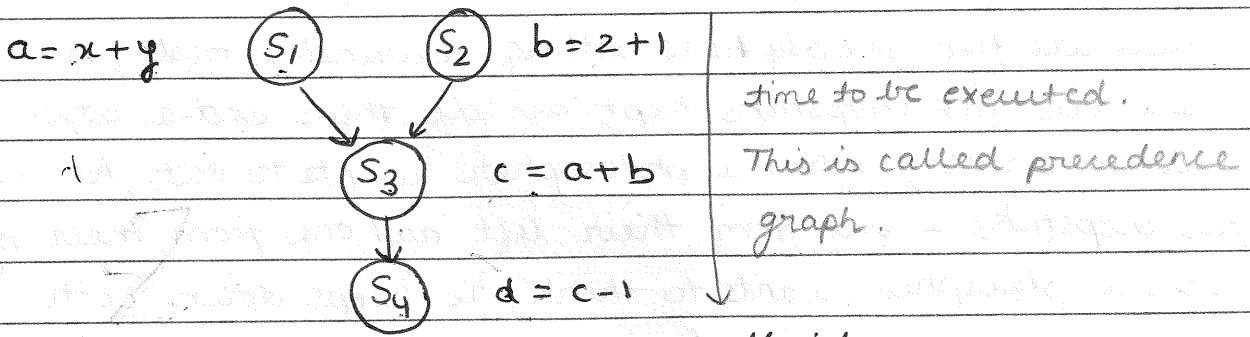
What is concurrent processing.

breaking a task into subtask, what are condition to be followed so that subtask can be processed concurrently (parallelly)

$S_1$ : $a = x + y$.
$S_2$ : $b = z + 1$.
$S_3$ : $c = a + b$.
$S_4$ : $d = c - 1$

} how many statement can be executed parallelly.

$S_1$ and $S_2$ are independent, so can run concurrently.




Two processes can be runned concurrently if

(i) $R(S_i) \cap R(S_j) \neq \phi$

(ii) $W(S_i) \cap R(S_j) = \phi$

(iii) $R(S_i) \cap W(S_j) = \phi$

(iv) $W(S_i) \cap W(S_j) = \phi$

where $S_i$ and $S_j$ are subtask and R() and W() are read and write set.

Dt.:
Pg.:

## classical Problem of Synchronization.

Following are some of the classical problem faced while process synchronization in systems where cooperating process are present

### Bounded Buffer Problem

- This problem is generalised in terms of Producer - Consumer problem.
- Solution to this problem is, creating two counting semaphores "full" and "empty" to keep track of current no. of full and empty buffers respectively.

### The Readers writers Problem

- In this problem there are some processes (called readers) that only read the shared data and never change it, and there are other processes (called writers) who may change the data in addition to reading or instead of reading it
- There are various type of reader - writers problem, most centred on relative priorities of readers and writers.

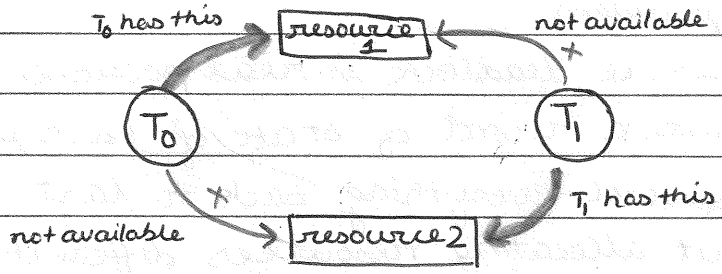
### Dining Philosophers Problem

- the dining philosopher's problem involves the allocation of limited resources from a group of processes in a deadlock-free and starvation free manner
- There are five philosophers sitting around a table, in which there are five chopsticks kept beside them and a bowl of rice in center, when a philosopher wants to eat, he uses two chopsticks - one from their left and one from their right. When a philospher wants to think, he keeps down both chopsticks at their original place.

Dt.:
Pg.:

## Deadlock.

Deadlock are a set of blocked processes each holding a resource and waiting to acquire a resource held by another process.




### How to prevent deadlocks. (preventing deadlocks)

Deadlock's can be avoided by avoiding atleast one of the four conditions, because all this four conditions are required simultaneously to cause deadlock. that means breaking anyone of the condition can avoid deadlock.

(i) Mutual Exclusion - Resources shared such as readonly files do not lead to deadlocks but resources, such as printers and tape drives, requires exclusive access by a single process. Resource that cannot be simultaneously shared by processes.

(ii) Hold and wait: Holding one or more resources while simultaneously waiting for one or more others.

(iii) No preemption: process can release resource at its own only. No one can preempt these resources.

(iv) Circular wait: waiting in circular.

$P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow P_3$

To avoid any one of the following condition can lead to avoidance of deadlock.

(i) mutual exclusion cannot be breaked. They should be accessed by single process at a time.

(ii) Hold and wait: No process can hold and request other resources

(iii) No preemption: No process can keep resources, we can snatch resources from process.

(iv) Circular wait: if we number all resources, and requires processes request resources only in strictly increasing (decreasing) order.

Dt.:
Pg.:

Handling deadlocks. (after deadlock has occured, how to handle them)

(i) Preemption - we can take a resource from one process and give to other. This will resolve the deadlock situation but sometime it does causes problem.

(ii) Rollback - In situations where deadlock is real possibility, system can periodically make a record of state of each process and when deadlock occurs, roll everything back to last checkpoint, and restart, but allocating resources differently so that deadlock doesnot occur.

(iii) kill one or more process : This is the simplest way, but it works. but can lead to starvation.

What is a Livelock ? (spinlock)

There is a variant deadlock called livelock. This is a situation in which two or more processes continuously change their state in response to changes in other process(es) without doing any useful work. This is a similar to deadlock in that no progress is made but differs in that neither process is blocked or waiting for anything.

A human example of livelock would be two people who meet face-to-face in corridor and each move aside to left the other pass, but they end up ~~saying~~ swaying from side to side without making any progress because they always move the same way at same time.